Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Übung 03: Git Übungen Tutorial

M.Sc. E. Trumann, M.Sc. J. Homann, Prof. Dr.-Ing. G. Payá Vayá,
7. November 2025

GIT Übungen

- Persönliches GIT Repository unter: `https://git.rz.tu-bs.de/theoretische-informatik/ti/lehre/HWSW/wise2526/`
- Frewillige Aufgaben zur Vorbereitung auf die Klausur
- Jede Woche Aufgaben zum aktuellen Thema in der Vorlesung/Übung
- Automatisch kontrolliert durch Testbenches in CI Pipeline
- Zwei Wege zur Bearbeitung:
  - Web: EDA Playground + Gitlab Webinterface
  - Lokal: VHDL Simulator + Git

GIT Übungen - Aufgabenblatt

Aufagbenblatt in Stud.IP, QR Code Links zu EDA Playground Template

Aufgabe VHDL-300: *Vier-1-Bit-Operand-Addierer*

Für diese Aufgabe soll eine vereinfachte Variante des `best_calculator`, der bereits aus der Präsenzübung bekannt ist, mit einer Fuktionalität gefüllt werden. Die Funktionalität soll dabei ausschließlich aus den grundlegenden Logikelementen (AND, OR, XOR, NAND, NOR, ...) und der Verkettung (&) erstellt werden.

Es sollen vier parallel arbeitende 1-Bit-Addierer implementiert werden. Das bedeutet, es sollen je 2 Bit der Eingabe (beispielsweise `data_bus(0)` und `data_bus(1)`, die darauf folgenden auch jeweils paarweise) addiert werden und in der Ausgabe `result(0)` und `result(1)` ausgegeben werden. Das Ergebnis für die weiteren Bits der Eingabe wird dann in die folgenden Bits der Ausgabe geschrieben, beispielsweise kommt das Ergebnis von `data_bus(6)` + `data_bus(7)` in `result(6)` und `result(7)`. Dabei soll im geringwertigeren Bit (im Beispiel `result(0)` und `result(6)`) das Ergebnis der Addition und im höherwertigen Bit (im Beispiel `result(1)` und `result(7)`) der entstandene Übertrag gespeichert werden.

https://edaplayground.com/x/pieA

VHDL-300

```vhdl
entity best_calculator is
port(
  data_bus     : in  std_ulogic_vector(7 downto 0);
  result       : out std_ulogic_vector(7 downto 0)
);
end best_calculator;
```

GIT Übungen - EDA Playground

Für jede Aufgabe gibt es ein Template als Ausgangspunkt

design.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;

entity best_calculator is
port(
    data_bus    : in  std_ulogic_vector(7 downto 0);
    result      : out std_ulogic_vector(7 downto 0)
);
end best_calculator;

-- TODO: Implement best_calculator (architecture)

```

GIT Übungen - Implementieren

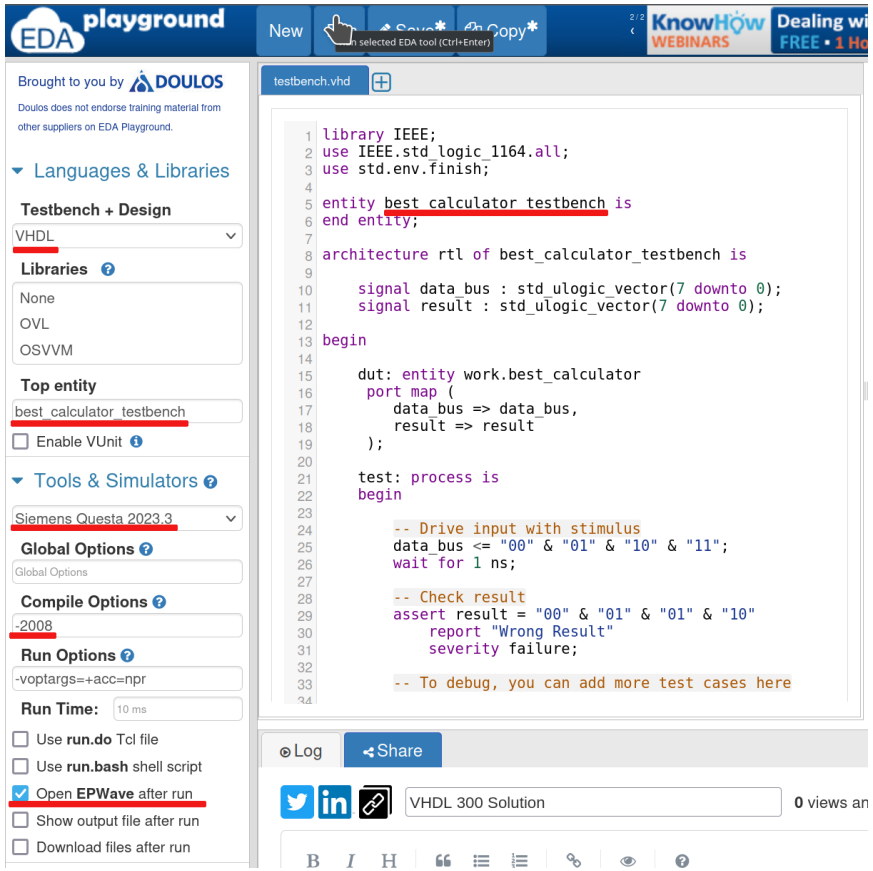
Ein Lösungsvorschlag für die erste Aufgabe

design.vhd

```
library IEEE;
use IEEE.std_logic_1164.all;

entity best_calculator is
port(
    data_bus    : in  std_ulogic_vector(7 downto 0);
    result      : out std_ulogic_vector(7 downto 0)
);
end best_calculator;

architecture rtl of best_calculator is
begin

    result(0) <= data_bus(0) xor data_bus(1);
    result(1) <= data_bus(0) and data_bus(1);

    result(2) <= data_bus(2) xor data_bus(3);
    result(3) <= data_bus(2) and data_bus(3);

    result(4) <= data_bus(4) xor data_bus(5);
    result(5) <= data_bus(4) and data_bus(5);

    result(6) <= data_bus(6) xor data_bus(7);
    result(7) <= data_bus(6) and data_bus(7);

end architecture;
```

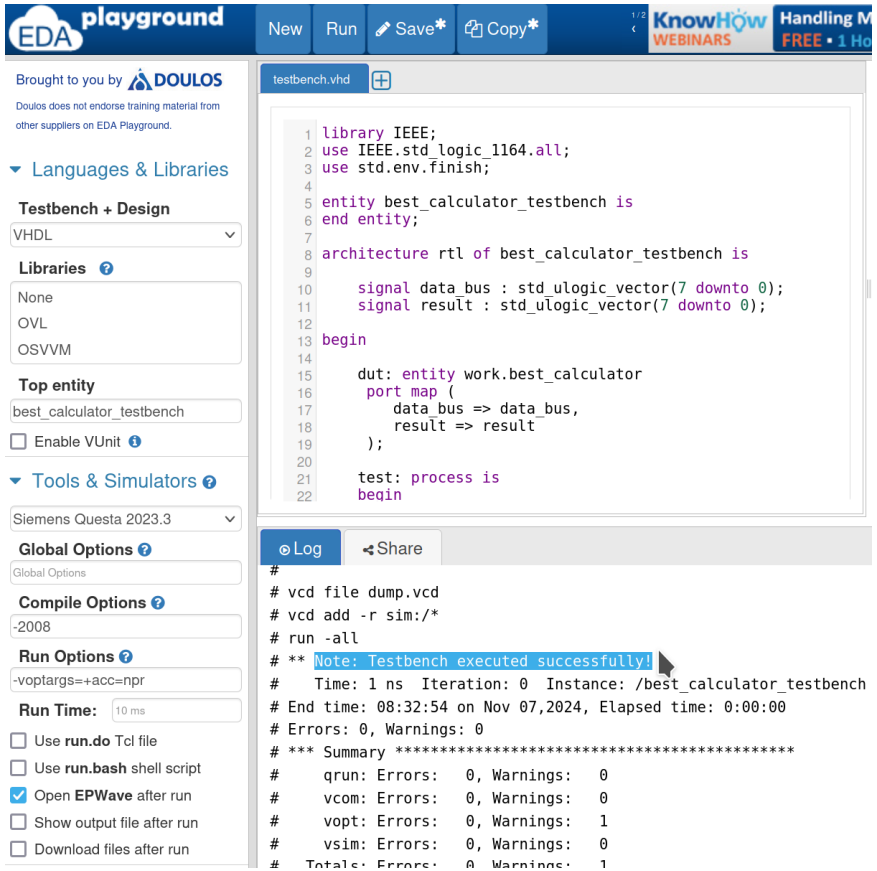
GIT Übungen - Testen

Die Lösung kann mit der mitgelieferten Testbench getestet werden



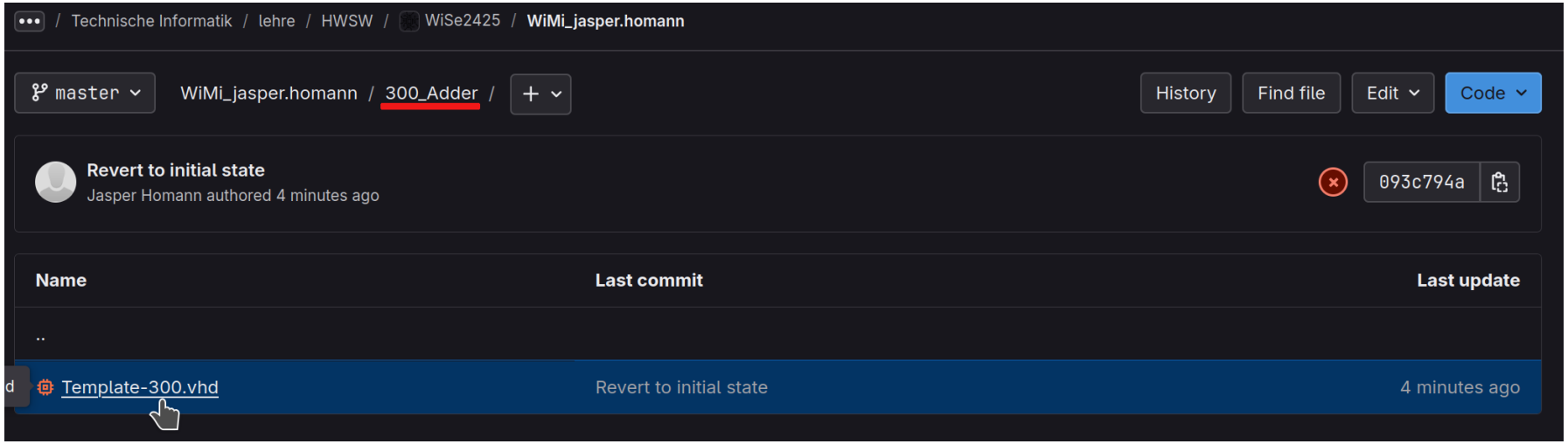
GIT Übungen - Fehler

Eventuelle Syntax und Laufzeitfehler werden im Log angezeigt



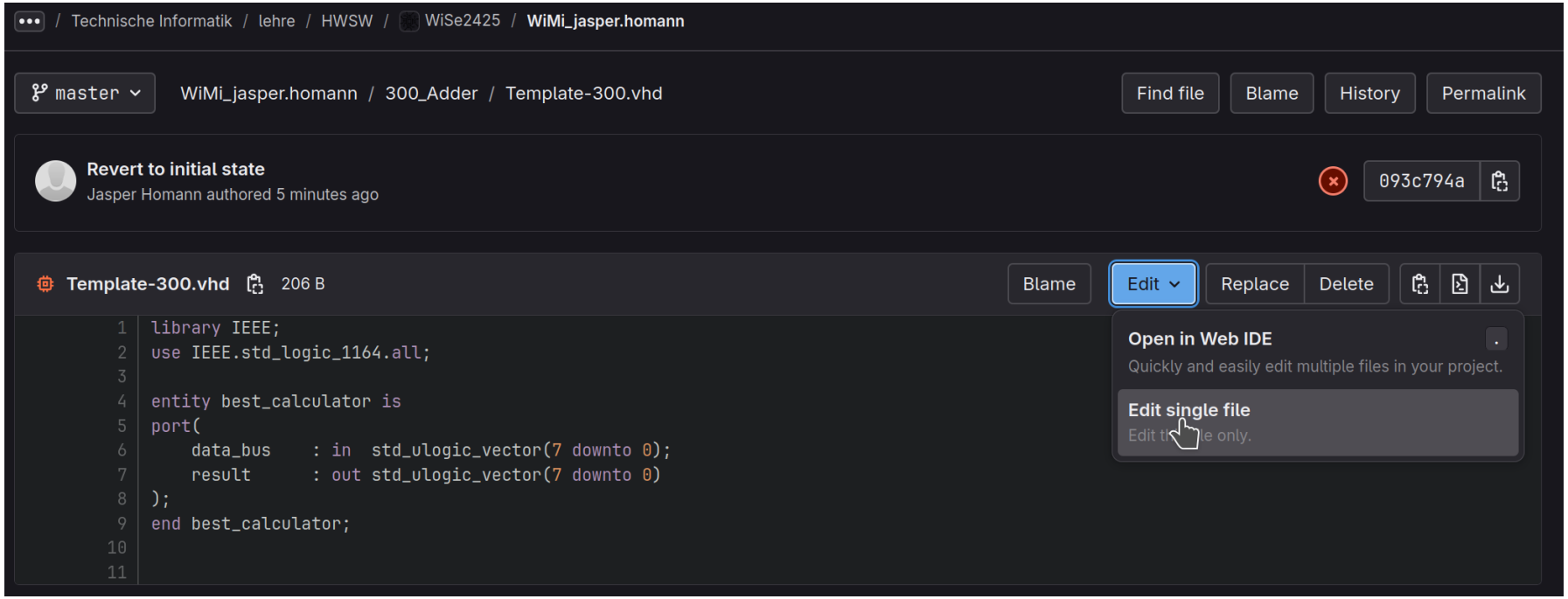
GIT Übungen - Gitlab

Template Datei im Gitlab öffnen



GIT Übungen - Gitlab

Template Datei im Gitlab bearbeiten



GIT Übungen - Lösung Übernehmen

Lösung aus EDA Playground kopieren und committen (speichern)

master | 300_Adder/Solution-300.vhd | No wrap

```
library IEEE;
use IEEE.std_logic_1164.all;

entity best_calculator is
port(
    data_bus    : in  std_ulogic_vector(7 downto 0);
    result      : out std_ulogic_vector(7 downto 0)
);
end best_calculator;

architecture rtl of best_calculator is
begin

    result(0) <= data_bus(0) xor data_bus(1);
    result(1) <= data_bus(0) and data_bus(1);

    result(2) <= data_bus(2) xor data_bus(3);
    result(3) <= data_bus(2) and data_bus(3);

    result(4) <= data_bus(4) xor data_bus(5);
    result(5) <= data_bus(4) and data_bus(5);

    result(6) <= data_bus(6) xor data_bus(7);
    result(7) <= data_bus(6) and data_bus(7);

end architecture;
```

Commit message

VHDL 300

Target Branch

master

Commit changes | Cancel

GIT Übungen - Überprüfen

CI Pipeline öffnen (Grüner Haken = Aufgabe Erfolgreich)

master | WiMi_jasper.homann / 300_Adder / **Solution-300.vhd** | Find file | Blame | History | Permalink

VHDL 300
Jasper Homann authored just now

4792d7e1

Solution-300.vhd 641 B | Blame | Edit | Replace | Delete

```
library IEEE;
use IEEE.std_logic_1164.all;

entity best_calculator is
port(
    data_bus    : in  std_ulogic_vector(7 downto 0);
    result      : out std_ulogic_vector(7 downto 0)
);
end best_calculator;

architecture rtl of best_calculator is
begin

    result(0) <= data_bus(0) xor data_bus(1);
    result(1) <= data_bus(0) and data_bus(1);

    result(2) <= data_bus(2) xor data_bus(3);
    result(3) <= data_bus(2) and data_bus(3);

    result(4) <= data_bus(4) xor data_bus(5);
    result(5) <= data_bus(4) and data_bus(5);

    result(6) <= data_bus(6) xor data_bus(7);
    result(7) <= data_bus(6) and data_bus(7);

end architecture;
```

GIT Übungen - Logs

Log Dateien auf Syntaxfehler und fehlgeschlagene Tests prüfen

Search visible log output

```
30  Errors: 0, Warnings: 0
31  Reading pref.tcl
32  # 2021.3
33  # vsim testbench_300
34  # vsim testbench_300
35  # Start time: 14:51:13 on Nov 07,2024
36  # ** Note: (vsim-3812) Design is being optimized...
37  # //  Questa Sim-64
38  # //  Version 2021.3 linux_x86_64 Jul 13 2021
39  # //
40  # //  Copyright 1991-2021 Mentor Graphics Corporation
41  # //  All Rights Reserved.
42  # //
43  # //  QuestaSim and its associated documentation contain trade
44  # //  secrets and commercial or financial information that are the property of
45  # //  Mentor Graphics Corporation and are privileged, confidential,
46  # //  and exempt from disclosure under the Freedom of Information Act,
47  # //  5 U.S.C. Section 552. Furthermore, this information
48  # //  is prohibited from disclosure under the Trade Secrets Act,
49  # //  18 U.S.C. Section 1905.
50  # //
51  # Loading std.standard
52  # Loading std.textio(body)
53  # Loading ieee.std_logic_1164(body)
54  # Loading ieee.numeric_std(body)
55  # Loading work.testbench_300(test300_arch)#1
56  #  run 4ns
57  # ** Note: 1 Test successful
58  #    Time: 1 ns  Iteration: 0  Instance: /testbench_300
59  # ** Note: 2 Test successful
60  #    Time: 2 ns  Iteration: 0  Instance: /testbench_300
61  # ** Note: 3 Test successful
62  #    Time: 3 ns  Iteration: 0  Instance: /testbench_300
63  # ** Note: 4 Test successful
64  #    Time: 4 ns  Iteration: 0  Instance: /testbench_300
65  #  exit
66  # End time: 14:51:14 on Nov 07,2024, Elapsed time: 0:00:01
67  # Errors: 0, Warnings: 0
68  Cleaning up project directory and file based variables        00:01
69  Job succeeded
```

GIT Übungen Lokal für Profis

- Empfehlung: Linux oder Windows + WSL (Beispiel: Debian/Ubuntu)
- Tools Installieren: `apt install git ghdl gtkwave`
- Repo Klonen: `git clone <URL>`
- VHDL Code Einlesen: `gdhl analyze --std=08 <Datei>`
- VHDL Testbench Elaborieren:
  `gdhl elaborate --std=08 <Entity>`
- VHDL Testbench ausführen:
  `ghdl run --std=08 <Entity> --wave=wave.ghw`
- Datei comitten und pushen
- CI im Web Interface anschauen
- Mithilfe von Ausgabe und GTKWave debuggen